

Name - Shiv Suryawanshi

PRN-202501050033 (CH2)

2.1.1. List operations

17:40

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options: 1. Add

2. Remove

3. Display

4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".

Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".

Display: Displays the current list of integers. If the list is empty, display "List is empty".

Quit: Exits the program.

The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Test case 1

1. •Add↵	
2. •Remove↵	
3. •Display↵	
4. •Quit↵	
Enter •choice: •1	

Integer:•11	
List•after•adding:•[11]↵	
1.•Add↵	
2.•Remove↵	
3.•Display↵	
4.•Quit↵	
Enter•choice:•1	
Integer:•25	
List•after•adding:•[11,•25]↵	
1.•Add↵	
2.•Remove↵	
3.•Display↵	
4.•Quit↵	
Enter•choice:•2	
Integer:•26	
Element•not•found↵	
1.•Add↵	
2.•Remove↵	
3.•Display↵	
4.•Quit↵	
Enter•choice:•2	
Integer:•11	
List•after•removing:•[25]↵	
1.•Add↵	

2. Remove ↵	
3. Display ↵	
4. Quit ↵	

Enter choice: 2	
Integer: 25	
List after removing: [] ↵	
1. Add ↵	
2. Remove ↵	
3. Display ↵	
4. Quit ↵	
Enter choice: 3	
List is empty ↵	
1. Add ↵	
2. Remove ↵	
3. Display ↵	
4. Quit ↵	
Enter choice: 2	
List is empty ↵	
1. Add ↵	
2. Remove ↵	
3. Display ↵	
4. Quit ↵	
Enter choice: 8	
Invalid choice ↵	

1. •Add↵	
2. •Remove↵	
3. •Display↵	
4. •Quit↵	
Enter •choice: •4	

Test case 2

1. •Add↵	
2. •Remove↵	
3. •Display↵	
4. •Quit↵	
Enter •choice: •1	
Integer: •1	
List •after •adding: •[1]↵	
1. •Add↵	
2. •Remove↵	
3. •Display↵	
4. •Quit↵	
Enter •choice: •1	
Integer: •2	
List •after •adding: •[1, •2]↵	
1. •Add↵	
2. •Remove↵	
3. •Display↵	
4. •Quit↵	
Enter •choice: •1	

Integer:·3	
List·after·adding:·[1,·2,·3]↵	
1.·Add↵	
2.·Remove↵	

3.·Display↵	
4.·Quit↵	
Enter·choice:·1	
Integer:·4	
List·after·adding:·[1,·2,·3,·4]↵	
1.·Add↵	
2.·Remove↵	
3.·Display↵	
4.·Quit↵	
Enter·choice:·2	
Integer:·2	
List·after·removing:·[1,·3,·4]↵	
1.·Add↵	
2.·Remove↵	
3.·Display↵	
4.·Quit↵	
Enter·choice:·2	
Integer:·6	
Element·not·found↵	
1.·Add↵	

2. • Remove↵	
3. • Display↵	
4. • Quit↵	
Enter • choice: • 3	
[1, • 3, • 4]↵	
1. • Add↵	
2. • Remove↵	
3. • Display↵	
4. • Quit↵	
Enter • choice: • 4	

Program code :

list=[] while

True:

```

print("1. Add") print("2.
Remove") print("3. Display")
print("4. Quit")
n=int(input("Enter choice: ")) if
n==1: i=int(input("Integer: "))
list.append(i) print(f"List after
adding: {list}") elif n==2: if
list==[]: print("List is empty")
else: i=int(input("Integer: "))
val=True for j in list:

```

```
if j==i:  
    list.remove(j) print(f"List after  
removing: {list}") val=False break  
if val:  
    print("Element not found")
```

```
elif n==3: if  
    list==[]:  
        print("List is empty") else:  
            print(list)  
elif n==4:  
    break else:  
        print("Invalid choice")
```

Average time
0.024 s
23.67 ms

Maximum time
0.033 s
33.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 23 ms Debug

Expected output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 25

List after adding: [11, 25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 11

List after removing: [25]

Actual output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 25

List after adding: [11, 25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 11

List after removing: [25]

Terminal Test cases

Debugger

Average time
0.024 s
23.67 ms

Maximum time
0.033 s
33.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 11

List after removing: [25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 25

List after removing: []

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

List is empty

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

List is empty

1. Add

2. Remove

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 11

List after removing: [25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 25

List after removing: []

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

List is empty

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

List is empty

1. Add

2. Remove

Terminal

Test cases

Debugger

Average time
0.024 s
23.67 ms

Maximum time
0.033 s
33.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 1 out of 1 hidden test case(s) passed

✓ Test case 2 15 ms Debug

Expected output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 1

List after adding: [1]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 2

List after adding: [1, 2]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 3

List after adding: [1, 2, 3]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 4

List after adding: [1, 2, 3, 4]

Actual output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 1

List after adding: [1]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 2

List after adding: [1, 2]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 3

List after adding: [1, 2, 3]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 4

List after adding: [1, 2, 3, 4]

Terminal

Test cases

Debugger

Average time
0.024 s
23.67 ms

Maximum time
0.033 s
33.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

```
3. Display
4. Quit
Enter choice: 1
Integer: 4
List after adding: [1, 2, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 2
List after removing: [1, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 6
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
[1, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

Debugger

2.1.2. Dictionary Operations

11:39

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

Insertion – Insert a new key-value pair into the dictionary using user input.

Update – Update the value of an existing key using user input.

Deletion – Delete a specified key from the dictionary using user input.

Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.

Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.

Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.

Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Test case 1

Original·Dictionary:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'Arjun',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali'}↵	
11	
Suresh	
After·Insertion:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'Arjun',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Suresh'}↵	
3	
Karthik	
After·Update:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·5:·'Arjun',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Suresh'}↵	
5	
After·Deletion:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Suresh'}↵	
Traversing·Dictionary:↵	
1:·Amit↵	
2:·Riya↵	
3:·Karthik↵	
4:·Neha↵	
6:·Pooja↵	
7:·Rahul↵	
8:·Sneha↵	
9:·Vikram↵	

10::Anjali↵	
11::Suresh↵	

Test case 2

Original·Dictionary::{1::'Amit',·2::'Riya',·3::'Kiran',·4::'Neha',·5::'Arjun',·6::'Pooja',·7::'Rahul',·8::'Sneha',·9::'Vikram',·10::'Anjali'}↵	
12	
Meera	
After·Insertion::{1::'Amit',·2::'Riya',·3::'Kiran',·4::'Neha',·5::'Arjun',·6::'Pooja',·7::'Rahul',·8::'Sneha',·9::'Vikram',·10::'Anjali',·12::'Meera'}↵	
6	
Divya	
After·Update::{1::'Amit',·2::'Riya',·3::'Kiran',·4::'Neha',·5::'Arjun',·6::'Divya',·7::'Rahul',·8::'Sneha',·9::'Vikram',·10::'Anjali',·12::'Meera'}↵	
1	
After·Deletion::{2::'Riya',·3::'Kiran',·4::'Neha',·5::'Arjun',·6::'Divya',·7::'Rahul',·8::'Sneha',·9::'Vikram',·10::'Anjali',·12::'Meera'}↵	
Traversing·Dictionary:↵	
2::Riya↵	
3::Kiran↵	
4::Neha↵	
5::Arjun↵	
6::Divya↵	
7::Rahul↵	
8::Sneha↵	
9::Vikram↵	
10::Anjali↵	

Program code: student

```
= {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}  
  
print("Original Dictionary:", student)  
key=int(input()) value=input()  
student[key]=value print("After  
Insertion:",student) key=int(input())  
newvalue=input()  
student[key]=newvalue print("After  
Update:",student) key=int(input())  
student.pop(key,None) print("After  
Deletion:", student) print("Traversing
```

Dictionary:") for key, value in

student.items():

print(f"{key} : {value}")

Average time

0.007 s

7.00 ms

Maximum time

0.011 s

11.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Debugger

Test case 11 ms Debug

Expected output

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

11

11

Suresh

Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

3

3

Karthik

Karthik

After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

5

5

After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

Traversing Dictionary:

1: Amit

1: Amit

2: Riya

2: Riya

3: Karthik

3: Karthik

4: Neha

4: Neha

6: Pooja

6: Pooja

7: Rahul

7: Rahul

8: Sneha

8: Sneha

9: Vikram

9: Vikram

10: Anjali

10: Anjali

11: Suresh

11: Suresh

Average time
0.007 s
7.00 ms

Maximum time
0.011 s
11.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Debugger

Test case 2 6 ms Debug

Expected output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

12

Meera

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

6

Divya

After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

1

After Deletion: {2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

Traversing Dictionary:

2: Riya

3: Kiran

4: Neha

5: Arjun

6: Divya

7: Rahul

8: Sneha

9: Vikram

10: Anjali

12: Meera

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

12

Meera

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

6

Divya

After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

1

After Deletion: {2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

Traversing Dictionary:

2: Riya

3: Kiran

4: Neha

5: Arjun

6: Divya

7: Rahul

8: Sneha

9: Vikram

10: Anjali

12: Meera

2.2.1. Linear search Technique

04:06

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

The first line of input contains the array of integers which are separated by space

The last line of input contains the key element to be searched

Output format:

If the element is found, print the index. If the element found multiple times, print the starting index If the element is not found, print Not found.

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

Output:

2

Test case 1

1 2 3 4 3 5 6	
3	
2	

Test case 2

1 2 3 4 5 6 7 8 9 19	
20	

Not • found	
-------------	--

```
arr = list(map(int,input().split()))
```

```
key = int(input()) found = False
```

```
for i in range(len(arr)):
```

```
    if arr[i] == key: print(i)
```

```
    found = True break
```

```
if not found: print("Not
```

```
found")
```

Average time
0.004 s
4.00 ms

Maximum time
0.005 s
5.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✕
Debugger

✓ Test case 1 4 ms Debug

Expected output

1 2 3 4 3 5 6

3

2

Actual output

1 2 3 4 3 5 6

3

2

✓ Test case 2 3 ms Debug

Expected output

1 2 3 4 5 6 7 8 9 19

20

Not found

Actual output

1 2 3 4 5 6 7 8 9 19

20

Not found

2.2.2. Captain of the Team

01:32

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Test case 1

171 169 185 156 174 191 186 190 187 172 160	
191↵	

Program code :

```
heights = list(map(int,input().split())) print(max(heights))
```

captainof...

Submit

Average time
0.003 s
2.67 ms

Maximum time
0.003 s
3.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 3 ms Debug

Expected output

171 169 185 156 174 191 186 190 187 172 160

191

Actual output

171 169 185 156 174 191 186 190 187 172 160

191

Terminal

Test cases

Debugger